

Beginner-C Bootcamp 8주차 과제

20215789 산업보안학과 김문정

Challenge 1. 단순 연결리스트 구현

```
===== 연결리스트 출력 =====
노드 주소 : 0x5dce00ff52a0 | data : 10 | next : 0x5dce00ff56d0
노드 주소 : 0x5dce00ff56d0 | data : 20 | next : 0x5dce00ff56f0
노드 주소 : 0x5dce00ff56f0 | data : 30 | next : 0x5dce00ff5710
노드 주소 : 0x5dce00ff5710 | data : 40 | next : 0x5dce00ff5730
노드 주소 : 0x5dce00ff5730 | data : 50 | next : (nil)
=====

[delete] 노드 삭제
삭제 데이터 : 30
삭제 노드 주소 : 0x5dce00ff56f0

===== 연결리스트 출력 =====
노드 주소 : 0x5dce00ff52a0 | data : 10 | next : 0x5dce00ff56d0
노드 주소 : 0x5dce00ff56d0 | data : 20 | next : 0x5dce00ff5710
노드 주소 : 0x5dce00ff5710 | data : 40 | next : 0x5dce00ff5730
노드 주소 : 0x5dce00ff5730 | data : 50 | next : (nil)
=====

[free] 해제 노드 주소 : 0x5dce00ff52a0
[free] 해제 노드 주소 : 0x5dce00ff56d0
[free] 해제 노드 주소 : 0x5dce00ff5710
[free] 해제 노드 주소 : 0x5dce00ff5730
user@user-VirtualBox:~$
```

각 노드의 주소와 그것이 가리키는 next 주소를 보면 연결리스트 형태로 구현된 것을 알 수 있다.

```

Breakpoint 1, append (head=0x7fffffffddd0, value=10) at linkedlist.c:13
13      Node* newNode = (Node*)malloc(sizeof(Node));
(gdb) n
15      if (newNode == NULL) {
(gdb) n
20      newNode->data = value;
(gdb) n
21      newNode->next = NULL;
(gdb) p newNode
$1 = (Node *) 0x5555555592a0
(gdb) p *newNode
$2 = {data = 10, next = 0x0}
(gdb) info proc mappings
process 3691
Mapped address spaces:

```

append에 bf를 걸고 newNode 주소를 출력해보면 0x5555555592a0이고 안에 있는 데이터는 10, 아직 아무것도 append되어있지 않으므로 0x0인 것을 확인할 수 있다.

info proc mappings로 확인해보면 heap주소가 0x555555559000 0x555555557a000인 것을 알 수 있는데 newNode주소가 0x5555555592a0 즉 heap범위 안에 있기 때문에 malloc을 하면 메모리주소가 heap영역에 할당된다는 것을 확인할 수 있다.

Start Addr	End Addr	Size	Offset	Perms	objfile
0x555555554000	0x555555555000	0x1000	0x0	r--p	/home/user
/linkedlist					
0x555555555000	0x555555556000	0x1000	0x1000	r-xp	/home/user
/linkedlist					
0x555555556000	0x555555557000	0x1000	0x2000	r--p	/home/user
/linkedlist					
0x555555557000	0x555555558000	0x1000	0x2000	r--p	/home/user
/linkedlist					
0x555555558000	0x555555559000	0x1000	0x3000	rw-p	/home/user
/linkedlist					
0x555555559000	0x555555557a000	0x21000	0x0	rw-p	[heap]
0x7ffff7c00000	0x7ffff7c28000	0x28000	0x0	r--p	/usr/lib/x

```

Breakpoint 1, append (head=0x7fffffffddd0, value=40) at linkedlist.c:13
13         Node* newNode = (Node*)malloc(sizeof(Node));
(gdb) c
Continuing.
[append] 새 노드 생성
      data = 40
      node 주소      : 0x555555559710
      next 주소      : (nil)

Breakpoint 1, append (head=0x7fffffffddd0, value=50) at linkedlist.c:13
13         Node* newNode = (Node*)malloc(sizeof(Node));
(gdb) p *head
$3 = (Node *) 0x5555555592a0
(gdb) p *head->next
$4 = {data = 20, next = 0x5555555596f0}
(gdb) p *head->next->next
$5 = {data = 30, next = 0x555555559710}
(gdb) p *head->next->next->next
$6 = {data = 40, next = 0x0}
(gdb)

```

continue로 append를 몇 번 더 수행해주고 p *head를 하면 맨 처음 저장한 data=10의 주소를 가리키고 있는 것을 볼 수 있다. 이후 p *head->next를 입력하면 20이 저장된 노드가 나오고, next에는 이 노드가 가리키고 있는 data=30의 주소가 출력되는 것을 확인할 수 있다. 다음으로 p *head->next->next를 입력하면 이전에 p *head->next로 가리킨 노드인 data=30인 노드가 나오는 것을 확인할 수 있고, next도 마찬가지로 data=30노드가 뒤에 올 것으로 가리키고 있는 data=40의 주소를 볼 수 있다. 마지막으로

p *head->next->next->next를 입력하면 맨 마지막에 있는 노드인 data=40을 가지고 있는 노드가 출력된 것을 확인할 수 있고 이것이 제일 끝에 있는 노드이므로 next를 보면 0x0인 것을 알 수 있다.

이를 통해

head

↓

0x92a0 [10 | next=0x92c0]

↓

0x92c0 [20 | next=0x92e0]

↓

0x92e0 [30 | next=NULL] 이런 식으로 next 포인터가 다음 노드 주소를 저장하면서 연결리스트가 만들어진다는 것을 알 수 있다.

Challenge2. 스택 구현

```
[push] 1 추가
      node 주소 : 0x5d27cc7e02a0
      next 주소 : (nil)

[push] 2 추가
      node 주소 : 0x5d27cc7e06d0
      next 주소 : 0x5d27cc7e02a0

[push] 3 추가
      node 주소 : 0x5d27cc7e06f0
      next 주소 : 0x5d27cc7e06d0

===== STACK TOP =====
주소 : 0x5d27cc7e06f0 | data : 3 | next : 0x5d27cc7e06d0
주소 : 0x5d27cc7e06d0 | data : 2 | next : 0x5d27cc7e02a0
주소 : 0x5d27cc7e02a0 | data : 1 | next : (nil)
=====

[peek] top 값 : 3
```

```
[peek] top 값 : 3

[pop] 3 제거
      삭제 노드 주소 : 0x5d27cc7e06f0

[pop] 2 제거
      삭제 노드 주소 : 0x5d27cc7e06d0

===== STACK TOP =====
주소 : 0x5d27cc7e02a0 | data : 1 | next : (nil)
=====

[pop] 1 제거
      삭제 노드 주소 : 0x5d27cc7e02a0

[pop] 스택이 비어있습니다 .
```

코드가 길어 실행 결과만 프린트하였는데, 위 사진과 같이 후입선출 구조로 프린트되는 것을 알 수 있다. push할 때 아래 코드와 같이 top을 newNode로 하고,

```
newNode->next = top;
```

```
top = newNode;
```

반대로 pop을 할때는 아래 코드와 같이 현재 top노드를 제거하고 새로운 노드를 top으로 제거해 가장 마지막에 들어온 데이터부터 제거되도록 구현하면 된다.

```
Node* temp = top;
```

```
top = top->next;
```

또한 프린트된 스택을 보면 현재 노드의 next가 다음 노드의 주소를 가리키고 있는 연결 리스트 구조인 것을 확인할 수 있다.

```
// pop
int pop() {
    Node* temp = top;
    int value = temp->data;

    top = top->next;

    printf("[pop] %d 제거\n", value);
    printf("삭제 노드 주소 : %p\n\n", (void*)temp);

    free(temp);

    return value;
}
```

```
[pop] 3 제거
삭제 노드 주소 : 0x616397a406f0

[pop] 2 제거
삭제 노드 주소 : 0x616397a406d0

===== STACK TOP =====
주소 : 0x616397a402a0 | data : 1 | next : (nil)
=====

[pop] 1 제거
삭제 노드 주소 : 0x616397a402a0
Segmentation fault (core dumped)
```

맨 위 사진(첫 번째 사진)에서는 `top == NULL` 일때를 정의해 스택이 비어있는 상태에서 `pop()`이 호출될 경우를 대비해 예외 처리를 추가하였다. 반면 세 번째 사진에서는 예외 처리를 삭제하고 스택이 비어 있는 상태에서 `pop()` 함수를 호출해 보았는데, 마지막 사진과 같이 Segmentation Fault, 허용되지 않은 메모리에 접근했다는 오류가 발생한다는 것을 확인할 수 있었다.